Task 1: Computer Vision using CNN Models

Overview: Establish a clean baseline, run controlled experiments (changing one variable at a time), and produce a comparative report styled like a mini research paper: hypothesis → method → results → analysis.

Global rules for Task 1

- One fixed random seed, set everywhere (NumPy, TensorFlow, Python)
- One fixed train / validation / test split, frozen and reused in every experiment
- A fixed training budget (same number of epochs) unless an experiment explicitly studies epoch count
- Every experiment logged in a results table — no result is valid unless it is reproducible and changes only one variable at a time

Part A — Traditional CNN Model (The Baseline)

Objective: Implement a traditional CNN using TensorFlow and Keras to classify CIFAR-10 images, establishing a clean, augmentation-free performance baseline that every later experiment is measured against.

Requirements — interns must:

- Load and preprocess the CIFAR-10 dataset; normalize pixel values
- Establish and lock the fixed random seed and train/validation/test split (frozen for the entire task)
- Build an AlexNet-style CNN adapted for small (32×32) images — adapt the architecture rather than copying the original 227×227 design
- Use convolutional layers with increasing filters (e.g., 64 → 128 → 256), ReLU activations, pooling layers, and fully connected layers
- Keep this a clean baseline — no data augmentation, no batch normalization, no dropout
- Train and evaluate the model

Expected output — interns must report:

- Model architecture and total parameter count
- Training time and number of training epochs (fixed budget for all experiments)
- Training, validation, and test accuracy
- Precision, Recall, F1-score
- Confusion matrix
- Research note: identify the 2–3 most-confused class pairs and form a hypothesis why (e.g. cat vs. dog similarity) — revisited in Part C

Success threshold: ≥ 70% test accuracy on CIFAR-10.

Part B — Controlled Experiments

1. Regularization study (diagnose and fix overfitting)

- Plot the baseline's training vs. validation curves and diagnose the overfitting gap
- Run separately: baseline + Dropout only; baseline + Batch Normalization only; baseline + L2 regularization only
- Report which single technique closed the train–val gap most, and whether any hurt accuracy

2. Data augmentation study (find the right amount)

- Run separately: light augmentation (horizontal flip); moderate (flip + small rotation/shift); aggressive (zoom, heavy rotation, brightness/contrast)
- Report whether aggressive augmentation helped or hurt, why it might distort 32×32 images, and recommend a level with justification

3. Optimization study (tune the training process)

- Optimizer comparison: SGD+momentum vs. Adam vs. RMSprop
- Learning-rate study: at least three rates (e.g. 0.01, 0.001, 0.0001), optionally with a scheduler
- Report best optimizer + learning-rate combination, with evidence

4. Architecture study (does more depth pay off?)

- Add one extra convolutional block and/or try a different kernel size
- Report accuracy gained vs. the extra parameters and training time it cost

Rules for fair comparison (all experiments)

- Same random seed and same train/test split as Part A
- Same epoch budget as Part A (if more epochs are used in any experiment, state and justify it)
- Each experiment isolates one change so its effect is attributable

Expected output: a single master experiment table — each row = one experiment, columns = test accuracy, train–val gap, parameter count, training time — plus a 1–2 sentence insight per experiment group.

Part C — Final Customized CNN

Objective: Combine the winning techniques identified in Part B into one customized CNN that measurably improves on the baseline — and justify every choice with your own experimental evidence.

Requirements — interns must:

- Build the final model using only the techniques Part B proved helpful (not everything at once)
- Re-test the Part A confusion hypothesis: did the most-confused class pairs improve, and by how much?

Success threshold: the custom CNN must beat the baseline's test accuracy by a measurable margin (target ≥ 3 percentage points), justified by experimental data — not random variation or extra training alone.

Expected output — interns must clearly explain:

- What changes were made and why each was chosen (citing the Part B experiment that justified it)
- How the changes improved the model, supported by the confusion-matrix comparison
- Final comparison between the baseline and the customized CNN, including:
  - Test accuracy (both models)
  - Parameter count and training time (both models)
  - Accuracy-vs-cost trade-off — accuracy gained per extra million parameters, with a final verdict

Deliverables for Task 1

Deliverable	Description
Python Code	Complete, reproducible CNN implementation (fixed seed, frozen split) + requirements.txt + README
Architecture Diagrams	Baseline and final customized CNN diagrams; training/validation accuracy and loss curves
Master Experiment Table	All Part B experiments in one table — test accuracy, train-val gap, parameter count, training time
Performance Table	Accuracy, Precision, Recall, F1-score for the baseline and the final model
Confusion Matrix	For both the baseline and the final customized

Deliverable	Description
	model, with most-confused class pairs identified
Trade-off Analysis	Accuracy-vs-cost comparison: accuracy gained per extra million parameters/training time, with a final verdict
Explanation	What changes were made, why each was chosen (citing the Part B experiment), and how they improved the model
Google Doc Report	Mini research-paper style: baseline → experiments → final model → trade-off → conclusion